🔄 Customer Churn Prediction — End-to-End ML Pipeline

> Predict which telecom customers will churn using a production-style ML pipeline with experiment tracking and REST API deployment.

Show Image

Show Image

Show Image

Show Image

Show Image

📌 Project Overview

This project goes beyond a Jupyter notebook. It's a **complete ML pipeline** built the way data science teams work in production:

- Real telecom dataset (7,043 customers, 21 features)
- Feature engineering to improve model signal
- 4 algorithms trained and compared with **MLflow experiment tracking**

- Best model deployed as a **live REST API** with FastAPI
- **SHAP explainability** to understand why the model predicts churn

Best model: Logistic Regression — ROC-AUC: **0.846**

📊 Results

Model	Accuracy	ROC-AUC	F1	Precision	Recall
Logistic Regression	0.8048	**0.8459**	0.5938	0.6634	0.5374
Gradient Boosting	0.8013	0.8423	0.5758	0.6643	0.5080
Random Forest	0.7921	0.8422	0.5527	0.6441	0.4840
XGBoost	0.7984	0.8410	0.5749	0.6531	0.5134

> **Why ROC-AUC?** The dataset is imbalanced (~26% churn). ROC-AUC measures how well the model ranks churners above non-churners regardless of class distribution — making it a more reliable metric than accuracy alone.

📐 Project Structure

```
churn-ml-pipeline/
├── data/
|   └── raw_churn.csv           # Auto-downloaded on first run
├── src/
|   ├── data_prep.py            # Data loading, cleaning, feature engineering
|   ├── train.py                # Model training + MLflow logging
|   └── evaluate.py             # Metrics, ROC curve, SHAP plots
├── api/
|   └── main.py                 # FastAPI prediction server
├── models/
|   ├── best_model.pkl          # Saved best model (auto-generated)
|   ├── feature_names.json      # Feature order for inference
|   └── plots/                  # Confusion matrix, ROC curve, SHAP
├── test_api.py                 # API smoke tests
├── requirements.txt
└── README.md
```

⚙️ Feature Engineering

Three new features were engineered beyond the raw dataset:

Feature	Description	Rationale
`tenure_group`	Tenure bucketed into 5 bands (0–1yr, 1–2yr, etc.)	Captures non-linear tenure effect
`charge_ratio`	TotalCharges ÷ (tenure + 1)	Normalised spending rate
`num_services`	Count of subscribed services	Engagement proxy — more services = less likely to churn

🚀 Quickstart

1. Clone and install

```bash
git clone https://github.com/YOUR_USERNAME/churn-ml-pipeline.git
cd churn-ml-pipeline
pip install -r requirements.txt
```

2. Train all models

```bash
python src/train.py
```

Output:

```
[logistic_regression] ROC-AUC=0.8459 | F1=0.5938
[gradient_boosting]   ROC-AUC=0.8423 | F1=0.5758
[random_forest]       ROC-AUC=0.8422 | F1=0.5527
[xgboost]             ROC-AUC=0.8410 | F1=0.5749

Best model saved → models/best_model.pkl
```

3. View MLflow experiment dashboard

```bash
mlflow ui
```

Open **http://localhost:5000** — compare all runs, metrics, and artifacts visually.

4. Generate evaluation plots

```bash
python src/evaluate.py
# Saves to models/plots/: confusion_matrix.png, roc_curve.png, shap_summary.png
```

5. Start prediction API

```bash
uvicorn api.main:app --reload
```

Open **http://localhost:8000/docs** for interactive Swagger UI.

🔌 API Usage

Predict churn for a customer

```bash
```

```
curl -X POST "http://localhost:8000/predict" \
  -H "Content-Type: application/json" \
  -d '{
    "gender": 0,
    "SeniorCitizen": 1,
    "Partner": 0,
    "Dependents": 0,
    "tenure": 2,
    "PhoneService": 1,
    "MultipleLines": 0,
    "InternetService": 1,
    "OnlineSecurity": 0,
    "OnlineBackup": 0,
    "DeviceProtection": 0,
    "TechSupport": 0,
    "StreamingTV": 0,
    "StreamingMovies": 0,
    "Contract": 0,
    "PaperlessBilling": 1,
    "PaymentMethod": 2,
    "MonthlyCharges": 75.50,
    "TotalCharges": 151.0,
    "tenure_group": 0,
    "charge_ratio": 75.5,
    "num_services": 2
  }'
```

Response

```json
{
  "churn_prediction": 1,
  "churn_probability": 0.7821,
  "risk_level": "HIGH",
  "message": "Customer is very likely to churn. Immediate retention action recommend
}
```

Available endpoints

Method	Endpoint	Description
GET	`/`	Health check
POST	`/predict`	Predict churn for one customer
GET	`/model-info`	Returns loaded model name and features
GET	`/docs`	Interactive Swagger UI

🔍 Key Insight from SHAP

After running SHAP analysis, the top 3 drivers of churn were:

1. **Contract type** — month-to-month customers churn at ~3× the rate of two-year contract customers
2. **Tenure** — customers in their first 12 months are the highest risk group
3. **Monthly charges** — customers paying above $70/month with short tenure are disproportionately likely to leave

This matches business intuition: new customers on flexible contracts paying high prices have the least switching cost.

🛠️ Tech Stack

Tool	Purpose
Pandas / NumPy	Data loading, cleaning, feature engineering
Scikit-learn	Model training, preprocessing, evaluation
XGBoost	Gradient boosted trees
MLflow	Experiment tracking, model registry
SHAP	Model explainability
FastAPI	REST API serving

Tool	Purpose
Pydantic	Input validation
Uvicorn	ASGI server
Joblib	Model serialization

📁 Dataset

IBM Telco Customer Churn — publicly available dataset with 7,043 telecom customers.

- 21 features: demographics, account info, subscribed services
- Target: `Churn` (Yes/No) — ~26% positive rate
- Auto-downloads on first run via `src/data_prep.py`

Source: IBM Sample Data Sets

👤 Author

Aryan Afzal

- LinkedIn: linkedin.com/in/aryan-afzal-316926369
- GitHub: github.com/YOUR_USERNAME

📄 License

MIT License — free to use, modify, and distribute.